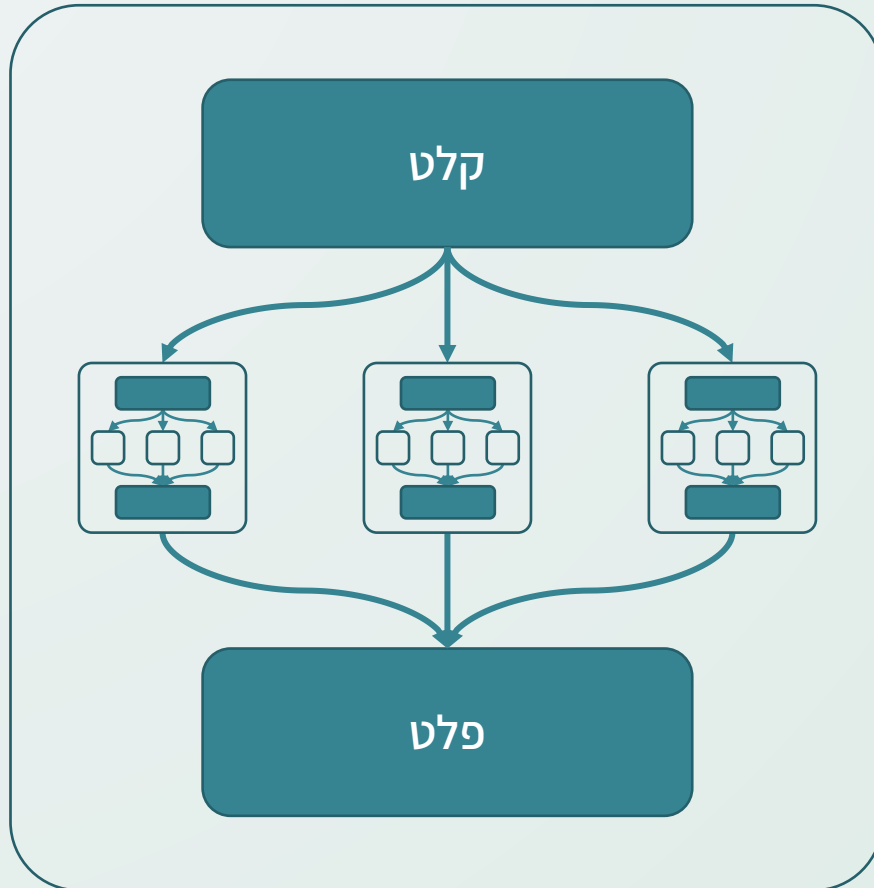


הפרד ומשול

תכנון וניתוח אלגוריתמים - תרגול שני
סמסטר תשפ"ו א' (אתגר)
המצגת באדיבות אלון קרימגנד

הפרד ומשול



- הפרד ומשול (Divide and Conquer) היא שיטה אלגוריתמית שבה פתרון לבעיה כלשהי:

1. **מחלק את הקלט** (בעיה) לחלק אחד או יותר פותר **באופן רקורסיבי** את הבעיה על החלקים השונים

3. **מרכיב את הפתרון** של הבעיה הכוללת מהפתרונות על החלקים

- כאשר אנו מנסחים אלגוריתם המשתמש בטכניקה, עלינו לא לשכוח:

1. להגדיר מהו **תנאי העצירה** של הרקורסיה. מהו מקרה הבסיס שבעבורו אנו פותרים את הבעיה ישירות?

2. להגדיר איך אנו **משלבים את הפתרונות** שהתקבלו לפתרון של הקלט הכולל.

מיון מיזוג

Merge Sort

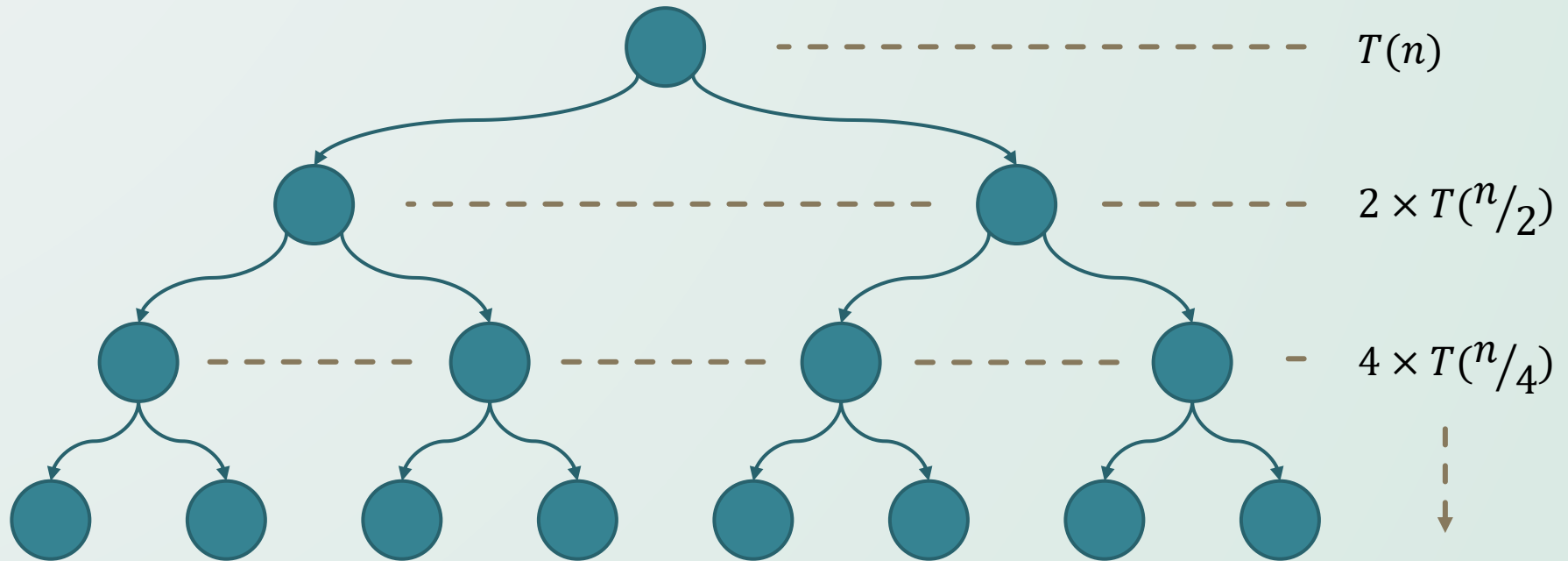
מיון מיזוג



- זהו אלגוריתם רקורסיבי המשתמש בטכניקת ההפרד ומשול.
- תחילה, בהינתן מערך, נחלק אותו לשני מערכים שונים, ונמייין אותם בצורה רקורסיבית - **הפרד**.
- המערך שמכיל איבר אחד הוא מערך ממוין: זהו תנאי העצירה!
- בהינתן שני המערכים הממוינים, נוכל לעבור עליהם בסיבוכיות לינארית ולמזג אותם למערך ממוין, כנדרש - **משול**.

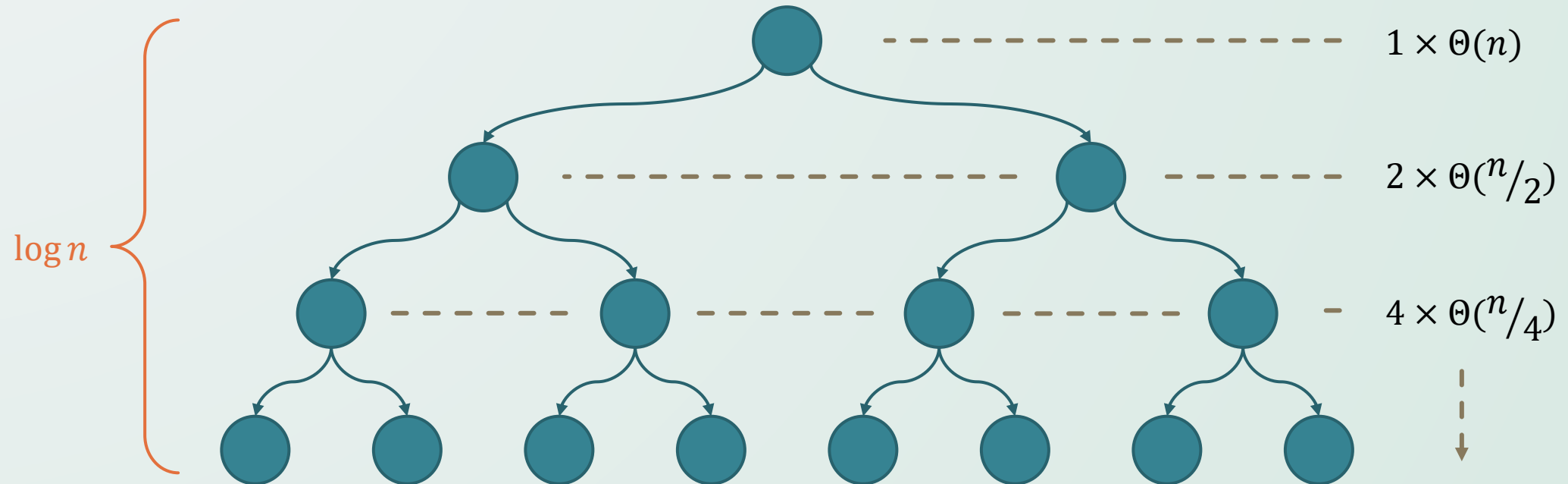
ניתוח סיבוכיות מיון המיזוג

- סיבוכיות מיון המיזוג נתונה ע"י הנוסחה הרקורסיבית $T(n) = 2T(n/2) + \Theta(n)$.
- נתבונן על עץ הרקורסיה:



ניתוח סיבוכיות מיון המיזוג

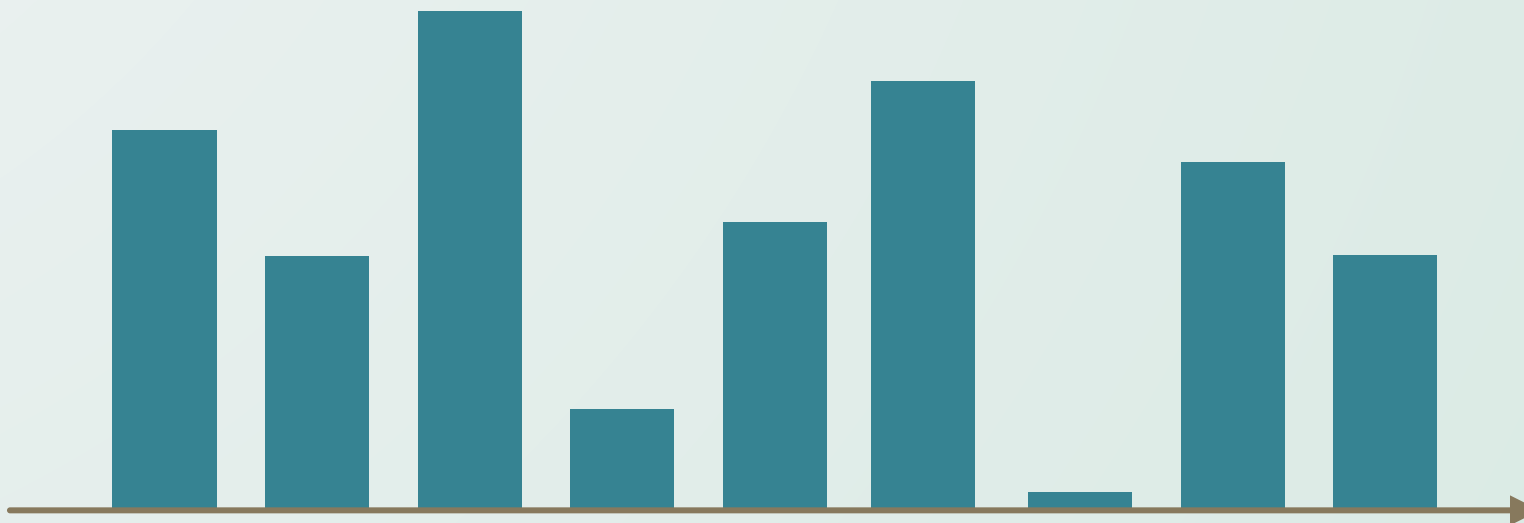
- בעץ הרקורסיה $\Theta(\log n)$ רמות, ובכל רמה מבוצעת סה"כ $\Theta(n)$ עבודה
- סיבוכיות ריצת כל האלגוריתם הינה אם כך $\Theta(n \log n)$, כנדרש.



בעיית הרווח המקסימלי

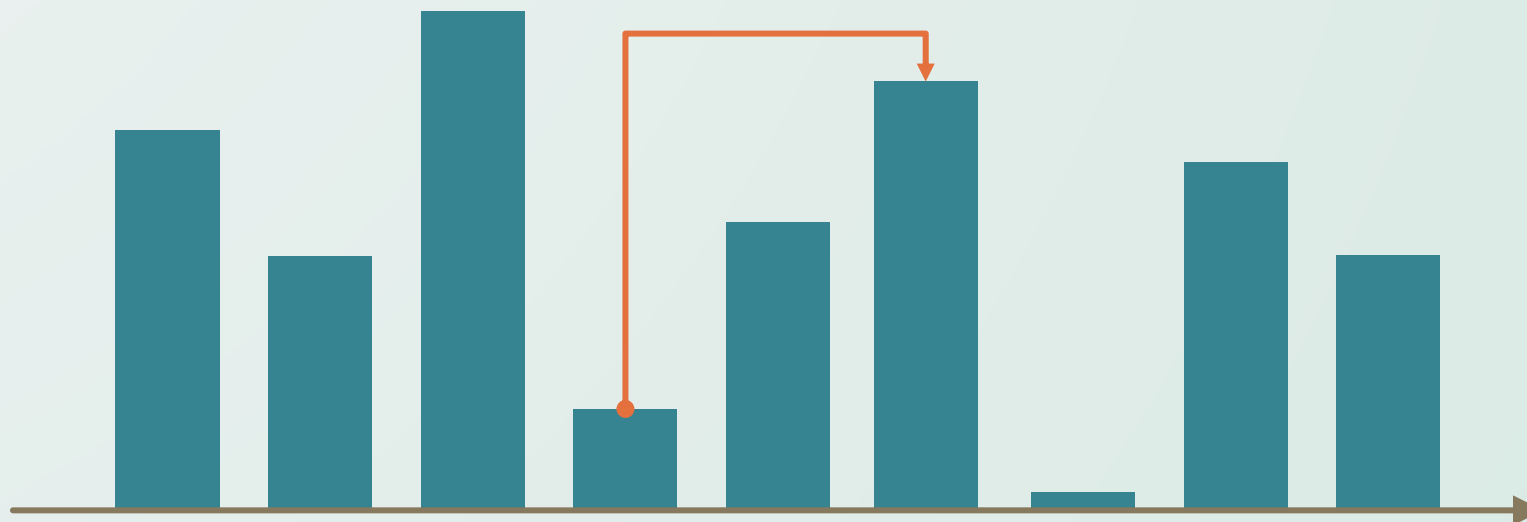
בעיית הרווח המקסימלי

- נתונה לכם רשימה של מחיר מנייה משוער ב־ n הימים הבאים $p_1, p_2, \dots, p_n$.
- אתם יכולים לקנות ולמכור את המנייה פעם אחת בלבד.
- תרצו לדעת באיזה יום לקנות i ולמכור j , כך ש־ $i < j$ והרווח $p_j - p_i$ מקסימלי.



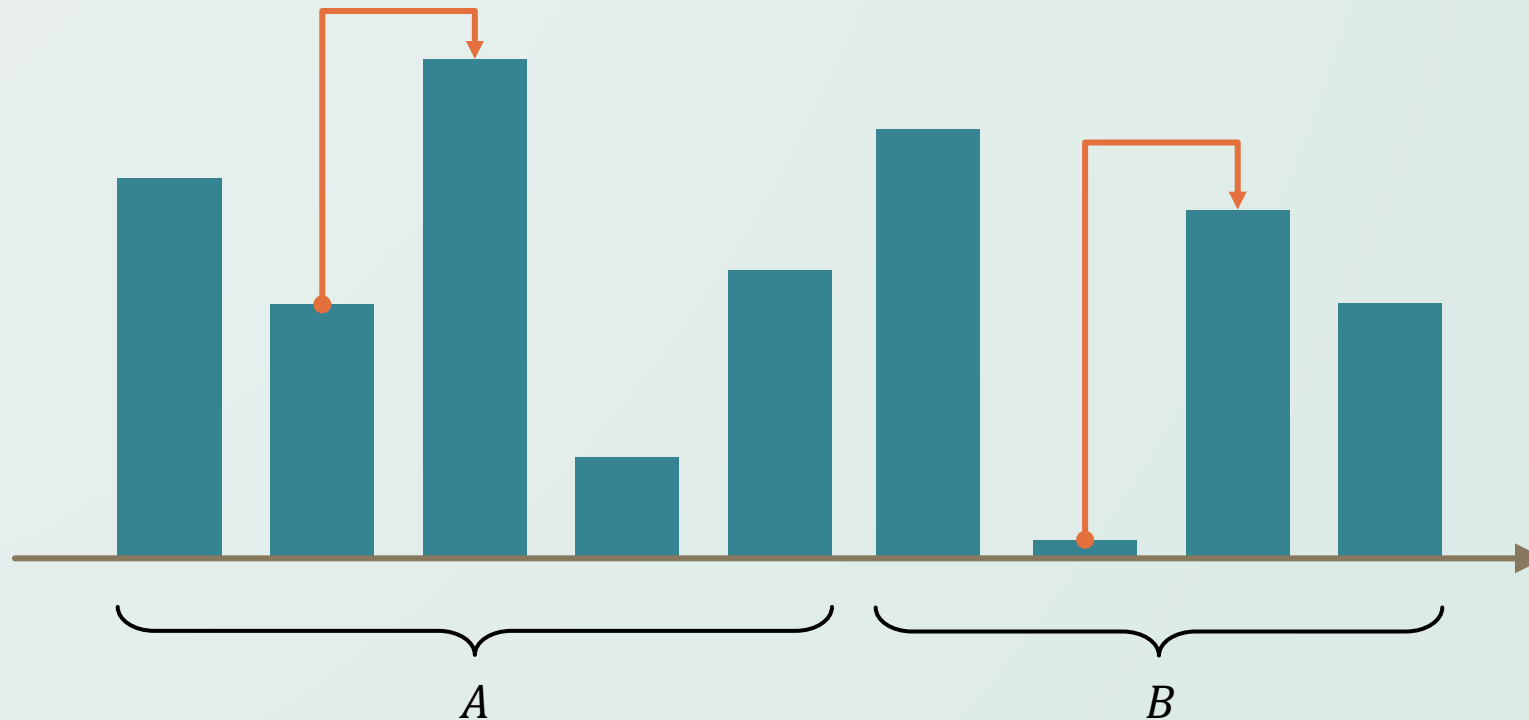
בעיית הרווח המקסימלי

- נתונה לכם רשימה של מחיר מנייה משוער ב־ n הימים הבאים $p_1, p_2, \dots, p_n$.
- אתם יכולים לקנות ולמכור את המנייה פעם אחת בלבד.
- תרצו לדעת באיזה יום לקנות i ולמכור j , כך ש־ $i < j$ והרווח $p_j - p_i$ מקסימלי.



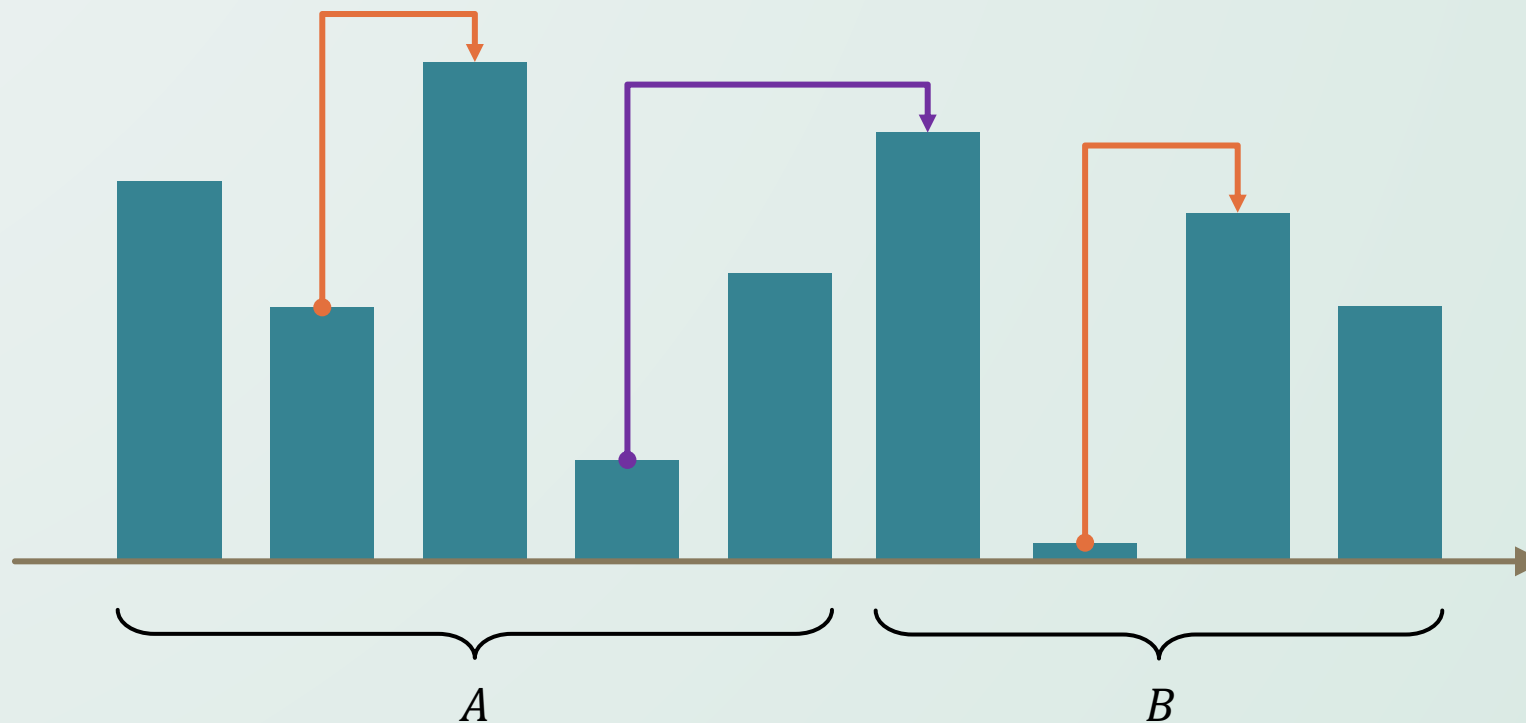
הבחנות בדרך לפתרון

- בבעיות שהקלט נתון בתור מערך מספרים, בדרך כלל פתרון הפרד ומשול יחלק את המערך לשני חצאים שווים A ו- B , ויפתור עליהם רקורסיבית.
- בהינתן הפתרון האופטימלי על $p_1, \dots, p_{\lceil n/2 \rceil}$ ועל $p_{\lceil n/2 \rceil + 1}, \dots, p_n$, מה נוכל להגיד על הפתרון הכולל?



הבחנות בדורך לפתרון

- הפתרון האופטימלי הכולל הוא:
- הפתרון האופטימלי ב- A , או הפתרון האופטימלי ב- B ,
- או הפתרון האופטימלי כאשר $i \in A$ ו- $j \in B$.



הפתרון

- בהינתן $p_1, p_2, \dots, p_n$ כקלט, **נפריד** אותו לשתי קבוצות A, B בגודל $\sim n/2$.
- חשוב שלכל $i \in A$ ו- $j \in B$ יתקיים $i < j$ (למה?)
- **נריץ את האלגוריתם רקורסיבי** על שתי הקבוצות A ו- B .
- הרקורסיה תעצור כאשר $n = 1$ ושם נחזיר שאין פתרון (לא ניתן להרוויח).

הפתרון

- בהינתן $p_1, p_2, \dots, p_n$ כקלט, **נפריד** אותו לשתי קבוצות A, B בגודל $\sim n/2$.
- חשוב שלכל $i \in A$ ו- $j \in B$ יתקיים $i < j$ (למה?)
- **נריץ את האלגוריתם רקורסיבי** על שתי הקבוצות A ו- B .
- הרקורסיה תעצור כאשר $n = 1$ ושם נחזיר שאין פתרון (לא ניתן להרוויח).
- נרצה למצוא את הפתרון האופטימלי שבו $i \in A$ ו- $j \in B$.
- פתרון זה ייקח את האיבר המינימלי ב- A ואת המקסימלי ב- B .
- נוכל למצוא מינימום ומקסימום בקבוצות אלו בזמן לינארי.

הפתרון

- בהינתן $p_1, p_2, \dots, p_n$ כקלט, **נפריד** אותו לשתי קבוצות A, B בגודל $\sim n/2$.
- חשוב שלכל $i \in A$ ו- $j \in B$ יתקיים $i < j$ (למה?)
- **נריץ את האלגוריתם רקורסיבי** על שתי הקבוצות A ו- B .
- הרקורסיה תעצור כאשר $n = 1$ ושם נחזיר שאין פתרון (לא ניתן להרוויח).
- נרצה למצוא את הפתרון האופטימלי שבו $i \in A$ ו- $j \in B$.
- פתרון זה ייקח את האיבר המינימלי ב- A ואת המקסימלי ב- B .
- נוכל למצוא מינימום ומקסימום בקבוצות אלו בזמן לינארי.
- הפתרון האופטימלי הכולל הוא **המקסימום** מבין שני הפתרונות הרקורסיביים והפתרון השלישי
- **סיבוכיות** הפתרון הינה $\mathcal{O}(n \log n)$ בדומה למיון מיזוג.
- זהו **לא הפתרון היעיל ביותר**. תוכלו לחשוב על פתרון בסיבוכיות $\mathcal{O}(n)$?

הפתרון המשופר

- נוכל להעביר ברקורסיה לא רק את הפתרון האופטימלי, אלא גם את המינימום והמקסימום בטווח.
- נרצה למצוא את הפתרון האופטימלי שבו $i \in A$ ו- $j \in B$.
- פתרון זה ייקח את האיבר המינימלי ב- A ואת המקסימלי ב- B .
- ערכים אלו יהיו זמינים לנו ב- $\mathcal{O}(1)$ מהרקורסיה!
- הפתרון האופטימלי הכולל הוא **המקסימום** מבין שני הפתרונות הרקורסיביים והפתרון השלישי.
- **סיבוכיות הפתרון הינה:**

$$T(n) = 2 \cdot T(n/2) + \mathcal{O}(1) \in \mathcal{O}(n)$$

הוכחת נכונות

- נוכל להוכיח את נכונות האלגוריתם באינדוקציה.
- **מקרה הבסיס:** כאשר $n = 1$ באמת אין פתרון ולא ניתן להרוויח מקנייה ומכירה של המניות.
- **צעד האינדוקציה:** נניח כי הוא נכון בעבור קלטים באורך $n > 1$ ונוכיח כי נכון בעבור n .
- נחלק את הבעיה לשתי קבוצות A, B כמו שעושה האלגוריתם.
- הפתרון האופטימלי הוא פתרון באחת מתתי הקבוצות, או פתרון כאשר $i \in A$ ו- $j \in B$.
- נוכל להשתמש בהנחת האינדוקציה ולהגיד שהפתרון שנקבל ברקורסיה על הקבוצות A, B אופטימלי.
- בעבור הפתרון מהסוג השלישי, ברור כי לקיחת i בתור המינימום ב- A ו- j בתור המקסימום ב- B ימקסם את הביטוי $p_j - p_i$ שכן $i < j$ בכל מקרה מתקיים.
- האלגוריתם יחזיר את המקסימום מבין שלושת הנציגים של האפשרויות, אך אפשרויות אלו מכסות את כל מרחב הפתרונות האפשרי, וכל אחד משלושת ה"נציגים" מהאפשרויות הוא האופטימלי. לכן הפתרון שיוחזר מקסימלי, כנדרש.

חזקה מהירה

Fast Exponentiation

חזקה מהירה

- פעולת הכפל היא פעולה בסיסית שמעבדים יודעים לבצע בזמן קבוע.
- עם זאת, העלאה בחזקה גבוהה היא פעולה יקרה יותר משמעותית.
- אם נרצה לחשב את a^b , האם ישנו אלגוריתם **שסיבוכיותו סאב-לינארית**?

$$\underbrace{a \cdot a \cdot a \cdot a \cdots a \cdot a \cdot a \cdot a}_{b \text{ times}}$$

הבחנות בדרך לפתרון

- נזכר בחוקי חזקות:
 $a^x \cdot a^y = a^{x+y}, (a^x)^y = a^{x \cdot y}$
- האם נוכל לחסוך חישובים מיותרים בדרך זו?

$$\underbrace{a \cdot a \cdot a \cdot a \cdots a \cdot a \cdot a \cdot a}_{b \text{ times}}$$

הפתרון

$$a^b = \begin{cases} (a^{b/2})^2 & \text{if } b \text{ is even} \\ (a^{\lfloor b/2 \rfloor})^2 \cdot a & \text{if } b \text{ is odd} \end{cases}$$

- בכל פעם אנו מחלקים את גודל המעריך ב-2 לפחות, תוך כדי ביצוע של $\mathcal{O}(1)$ פעולות.
- לכן סיבוכיות הפתרון הינה:
 $T(n) = T(n/2) + \mathcal{O}(1) \in \mathcal{O}(\log n)$

$$\underbrace{a \cdot a \cdot a \cdot a \cdots a \cdot a \cdot a \cdot a}_{b/2 \text{ times}} \cdot \underbrace{a \cdot a \cdot a \cdot a}_{b/2 \text{ times}}$$

נכונות

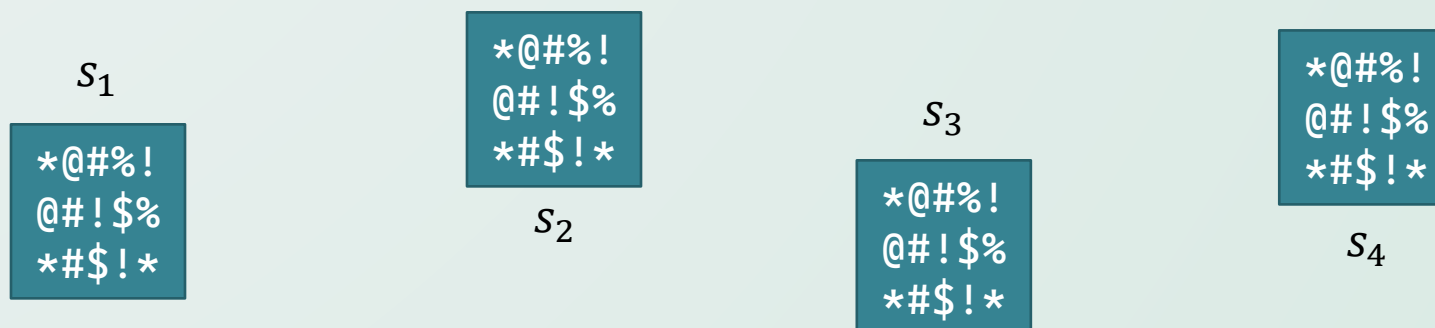
- באינדוקציה.
- בסיס: $n=1$: $a=a$
- ה.א.: נניח כי מתקיים הביטוי בעבור $1 \leq i \leq n$ ונוכיח בעבור n .
- אם n זוגי: $(a^{\frac{n}{2}})^2$
- אם n אי זוגי: $a * (a^{\lfloor \frac{n}{2} \rfloor})^2$

בעיית האיבר המרובה

Majority Element

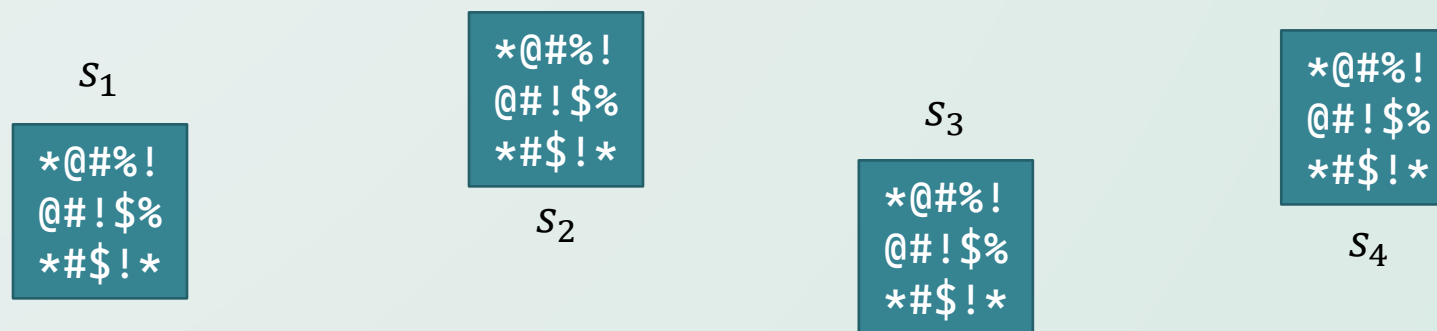
בעיית האיבר המרובה

- ברשותכם קבוצה של n מחרוזות מקודדות שונות $S = \{s_1, s_2, \dots, s_n\}$ המייצגות משאבים שנמצאים בשימוש על ידי משתמשים במערכת.
- מכיוון שהמחרוזות מקודדות, במבט ראשוני הן לא אומרות כלום ונראות כמו ג'יבריש מוחלט.



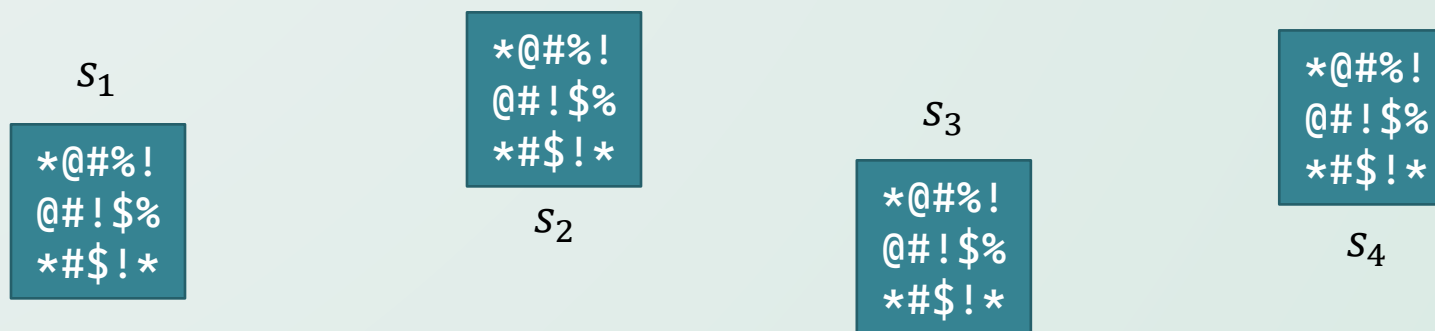
בעיית האיבר המרובה

- ברשותכם קבוצה של n מחרוזות מקודדות שונות $S = \{s_1, s_2, \dots, s_n\}$ המייצגות משאבים שנמצאים בשימוש על ידי משתמשים במערכת.
- מכיוון שהמחרוזות מקודדות, במבט ראשוני הן לא אומרות כלום ונראות כמו ג'יבריש מוחלט.
- ברשותכם אלגוריתם יעיל $A(\tilde{s}, \hat{s})$ אשר יודע לקבל שתי מחרוזות (משאבים) ולהבין האם הן שייכות לאותו המשתמש, בסיבוכיות $\mathcal{O}(1)$.
- תארו אלגוריתם, בסיבוכיות $\mathcal{O}(n \log n)$, שישתמש ב- A ויקבע האם ישנו משתמש אשר מעמיס על המערכת ומשתמש ביותר מחצי מהמשאבים.



כמה מילים לפני הפתרון

- כמו בבעיית הרווח המקסימלי, הפתרון הטרוויאלי הינו לעבור על $\mathcal{O}(n^2)$ זוגות האיברים האפשריים ולהשוות ביניהם.
- לאלגוריתמים רבים המשתמשים בהפרד ומשול יש תכונה זו.
- הפרד ומשול מצליח לעיתים קרובות לחסוך את כמות ההשוואות ולהוריד אותה ל- $\mathcal{O}(n \log n)$.
- בנוסף, ברור כי אנו לא באמת צריכים להשוות כל זוג איברים.
- הרי יחס השוויון הוא טרנזיטיבי: אם $a = b$ וגם $b = c$ אזי $a = c$ וחסכנו השוואה!



הפתרון

- נחלק את הקלט לשתי קבוצות בגודל $n/2 \sim$, ונסמן $A = \{s_1, \dots, s_{\lceil n/2 \rceil}\}, B = \{s_{\lceil n/2 \rceil+1}, \dots, s_n\}$.
- אם ישנו איבר שמייצג רוב ב- S , הרי שהוא חייב להיות רוב גם לפחות באחת מהקבוצות A, B (למה?)
- נסמן בתור $f(A)$ את האיבר שמופיע יותר מ- $n/2$ פעמים ב- A . באופן דומה נסמן $f(B)$.
- כעת, יש לנו לכל היותר שני מועמדים להיות $f(S)$ והם $f(A)$ ו- $f(B)$ בלבד.

הפתרון

- נחלק את הקלט לשתי קבוצות בגודל $n/2 \sim$, ונסמן $A = \{s_1, \dots, s_{\lfloor n/2 \rfloor}\}, B = \{s_{\lfloor n/2 \rfloor + 1}, \dots, s_n\}$.
- אם ישנו איבר שמייצג רוב ב- S , הרי שהוא חייב להיות רוב גם לפחות באחת מהקבוצות A, B (למה?)
- נסמן בתור $f(A)$ את האיבר שמופיע יותר מ- $n/2$ פעמים ב- A . באופן דומה נסמן $f(B)$.
- כעת, יש לנו לכל היותר שני מועמדים להיות $f(S)$ והם $f(A)$ ו- $f(B)$ בלבד.
- כדי לבדוק האם $\mathcal{O}(1)$ מהמועמדים הם באמת רוב, נוכל פשוט להשוות אותם לכל שאר האיברים ב- S בסיבוכיות $\mathcal{O}(n)$.
- אם אף אחד מהמועמדים לא קיבל רוב, פשוט נחזיר כי אין רוב ב- S .
- סיבוכיות הפתרון הכוללת אם כך הינה:
$$T(n) = 2 \cdot T(n/2) + \mathcal{O}(n) \in \mathcal{O}(n \log n)$$
- לבעיה זו יש אלגוריתם הפותר אותה בזמן לינארי. חפשו **Boyer-Moore Algorithm**.

קידוד הופמן וקודים חסרי רישא

Huffman Coding and prefix-free codes

קודים חסרי רישא

- **הבעיה** – בהינתן טקסט T מעל A^n $\Sigma = \{a_1, a_2, \dots, a_n\}$ נרצה לקודד אותו באמצעות A^n בבינארי.
- **הגבלות** – בהינתן הקידוד, כמובן שצריך לדעת לפענח בחזרה את T . בנוסף, נרצה קידוד ופענוח פולינומיאליים.

קודים חסרי רישא

- **הבעיה** – בהינתן טקסט T מעל אלפבית $\Sigma = \{a_1, a_2, \dots, a_n\}$ נרצה לקודד אותו באמצעות אלפבית בינארי.
- **הגבלות** – בהינתן הקידוד, כמובן שצריך לדעת לפענח בחזרה את T . בנוסף, נרצה קידוד ופענוח פולינומיאליים.
- **פונקציית המטרה** – נרצה שהקידוד יהיה הקצר ביותר האפשרי.

דוגמאות לקידודים

- נניח שיש לנו את הטקסט $T = abbca$, כמובן ש $\Sigma = \{a,b,c\}$. נסתכל לדוגמא בקידוד הבא:

דוגמאות לקידודים

- נניח שיש לנו את הטקסט $T = abbca$, כמובן ש $\Sigma = \{a,b,c\}$. נסתכל לדוגמא בקידוד הבא:

a	b	c
00	0	1

- אזי הטקסט המקודד יהיה $T' = 0000100$. אוי ואבוי לנו אם זה המצב! כי הרי אפשר לפענח את T' בתור הטקסט המקורי, אבל גם בתור $T = aacbb$. לכן הקידוד הזה **לא תקין**.

דוגמאות לקידודים

- נניח שיש לנו את הטקסט $T = abbca$, כמובן ש $\Sigma = \{a,b,c\}$. נסתכל לדוגמא בקידוד הבא:

a	b	c
00	0	1

- אזי הטקסט המקודד יהיה $T' = 0000100$. אוי ואבוי לנו אם זה המצב! כי הרי אפשר לפענח את T' בתור הטקסט המקורי, אבל גם בתור $T = aacbb$. לכן הקידוד הזה **לא תקין**.
- נסתכל על הקידוד הבא:

a	b	c
00	01	1

- נוכל להשתכנע שזה כן קידוד תקין, עם קצת ניסויים.

קודים חסרי רישא

- הסיבה שהקוד האחרון היה תקין, הוא שזה קוד **חסר רישות**.
- **הגדרה** - קוד $C = \{c_1, c_2, \dots, c_n\}$ הוא קבוצה של מחרוזות, לא בהכרח באורך קבוע. קוד C יקרא חסר רישות, אם אין מילה c_i שהיא רישה של מילה אחרת c_j .

קודים חסרי רישא

- הסיבה שהקוד האחרון היה תקין, הוא שזה קוד **חסר רישות**.
- **הגדרה** - קוד $C = \{c_1, c_2, \dots, c_n\}$ הוא קבוצה של מחרוזות, לא בהכרח באורך קבוע. קוד C יקרא חסר רישות, אם אין מילה c_i שהיא רישה של מילה אחרת c_j .
- **טענה** - קודים חסרי רישות ניתנים לפענוח יחיד. נוכל להוכיח זאת באינדוקציה.

קודים חסרי רישא

- הסיבה שהקוד האחרון היה תקין, הוא שזה קוד **חסר רישות**.
- **הגדרה** - קוד $C = \{c_1, c_2, \dots, c_n\}$ הוא קבוצה של מחרוזות, לא בהכרח באורך קבוע. קוד C יקרא חסר רישות, אם אין מילה c_i שהיא רישה של מילה אחרת c_j .
- **טענה** - קודים חסרי רישות ניתנים לפענוח יחיד. נוכל להוכיח זאת באינדוקציה.
- יהי $T = a_1 a_2 \dots a_n$ מעל א"ב $\Sigma = \{\sigma_1 \dots \sigma_k\}$, ונקודד אותו לטקסט הבינארי $T' = b_1 \dots b_m$.
- תנאי בסיס - אם $T' = \varepsilon$ נפענח אותו לטקסט הריק וברור שזה הפענוח היחיד. (האם באמת ברור?)

קודים חסרי רישא

- הסיבה שהקוד האחרון היה תקין, הוא שזה קוד **חסר רישות**.
- **הגדרה** - קוד $C = \{c_1, c_2, \dots, c_n\}$ הוא קבוצה של מחרוזות, לא בהכרח באורך קבוע. קוד C יקרא חסר רישות, אם אין מילה c_i שהיא רישה של מילה אחרת c_j .
- **טענה** - קודים חסרי רישות ניתנים לפענוח יחיד. נוכל להוכיח זאת באינדוקציה.
- יהי $T = a_1 a_2 \dots a_n$ מעל א"ב $\Sigma = \{\sigma_1 \dots \sigma_k\}$, ונקודד אותו לטקסט הבינארי $T' = b_1 \dots b_m$.
- תנאי בסיס - אם $T' = \varepsilon$ נפענח אותו לטקסט הריק וברור שזה הפענוח היחיד. (האם באמת ברור?)
- נפענח כעת את התו הראשון בד - נעבור על T' עד שרישא כלשהי שלו מתאימה לקידוד של תו כלשהו σ בא"ב. נזכור את התו, ונפענח כעת את הטקסט הקטן יותר. (כמובן בשביל פורמליות מלאה נצטרך להראות את צעד האינדוקציה ואז להפעיל אותו על הטקסט הקטן יותר).

קודים חסרי רישא

- הסיבה שהקוד האחרון היה תקין, הוא שזה קוד **חסר רישות**.
- **הגדרה** - קוד $C = \{c_1, c_2, \dots, c_n\}$ הוא קבוצה של מחרוזות, לא בהכרח באורך קבוע. קוד C יקרא חסר רישות, אם אין מילה c_i שהיא רישה של מילה אחרת c_j .
- **טענה** - קודים חסרי רישות ניתנים לפענוח יחיד. נוכל להוכיח זאת באינדוקציה.
- יהי $T = a_1 a_2 \dots a_n$ מעל א"ב $\Sigma = \{\sigma_1 \dots \sigma_k\}$, ונקודד אותו לטקסט הבינארי $T' = b_1 \dots b_m$.
- תנאי בסיס - אם $T' = \varepsilon$ נפענח אותו לטקסט הריק וברור שזה הפענוח היחיד. (האם באמת ברור?)
- נפענח כעת את התו הראשון בד - נעבור על T' עד שרישא כלשהי שלו מתאימה לקידוד של תו כלשהו σ בא"ב. נזכור את התו, ונפענח כעת את הטקסט הקטן יותר. (כמובן בשביל פורמליות מלאה נצטרך להראות את צעד האינדוקציה ואז להפעיל אותו על הטקסט הקטן יותר).
- **שאלה** - למה לא יתכן שנטעה בפענוח התו הראשון?

קודים חסרי רישא

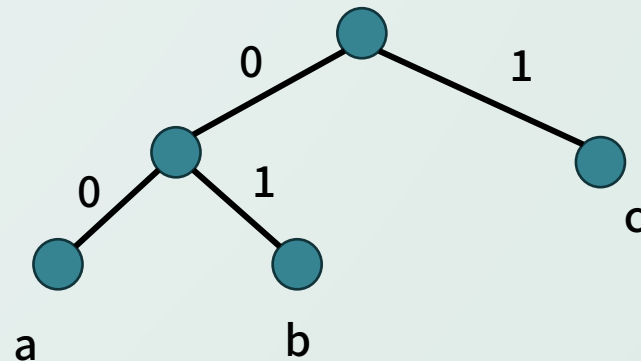
- הסיבה שהקוד האחרון היה תקין, הוא שזה קוד **חסר רישות**.
- **הגדרה** - קוד $C = \{c_1, c_2, \dots, c_n\}$ הוא קבוצה של מחרוזות, לא בהכרח באורך קבוע. קוד C יקרא חסר רישות, אם אין מילה c_i שהיא רישה של מילה אחרת c_j .
- **טענה** - קודים חסרי רישות ניתנים לפענוח יחיד. נוכל להוכיח זאת באינדוקציה.
- יהי $T = a_1 a_2 \dots a_n$ מעל א"ב $\Sigma = \{\sigma_1 \dots \sigma_k\}$, ונקודד אותו לטקסט הבינארי $T' = b_1 \dots b_m$.
- תנאי בסיס - אם $T' = \varepsilon$ נפענח אותו לטקסט הריק וברור שזה הפענוח היחיד. (האם באמת ברור?)
- נפענח כעת את התו הראשון בד - נעבור על T' עד שרישא כלשהי שלו מתאימה לקידוד של תו כלשהו σ בא"ב. נזכור את התו, ונפענח כעת את הטקסט הקטן יותר. (כמובן בשביל פורמליות מלאה נצטרך להראות את צעד האינדוקציה ואז להפעיל אותו על הטקסט הקטן יותר).
- **שאלה** - למה לא יתכן שנטעה בפענוח התו הראשון?
- **תשובה** - זה קוד חסר רישות, לכן אם התו הראשון היה אחר, הרי שקיבלנו ש σ מקודדת בתור רישא של תו אחר, **סתירה**.

קידוד הופמן

- ראינו שקודים חסרי רישות ניתנים תמיד לפענוח יחיד. נרצה למצוא את הקידוד האופטימלי בעזרת קודים חסרי רישות.
- נשים לב שאפשר למפות קודים חסרי רישות לעצים בינאריים ולהיפך. כל עלה ייצג תו, והמסלול מייצג את הקידוד – תזוזה שמאלה זה 0, ימינה 1.

קידוד הופמן

- ראינו שקודים חסרי רישות ניתנים תמיד לפענוח יחיד. נוכיח שהם מסוגלים להגיע לאופטימום מבחינת אורך הקידוד.
- נשים לב שאפשר למפות קודים חסרי רישות לעצים בינאריים ולהיפך. כל עלה ייצג תו, והמסלול מייצג את הקידוד – תזוזה שמאלה זה 0, ימינה 1.
- לדוגמא עבור הקידוד האחרון שראינו ($a \rightarrow 00, b \rightarrow 01, c \rightarrow 1$) נקבל את העץ:



קידוד הופמן

- נחזור לטקסט T . לכל $\sigma \in \Sigma$ נגדיר f_σ בתור התדירות של התו בטקסט. כלומר מספר ההופעות לחלק לאורך הטקסט. פונקציית המטרה שאנחנו רוצים למזער היא $\sum_{\sigma \in \Sigma} f_\sigma \cdot |c(\sigma)|$, כאשר $c(\sigma)$ זה הקידוד הבינארי.

קידוד הופמן

- נחזור לטקסט T . לכל $\sigma \in \Sigma$ נגדיר f_σ בתור התדירות של התו בטקסט. כלומר מספר ההופעות לחלק לאורך הטקסט. פונקציית המטרה שאנחנו רוצים למזער היא $\sum_{\sigma \in \Sigma} f_\sigma \cdot |c(\sigma)|$, כאשר $c(\sigma)$ זה הקידוד הבינארי.
- נסתכל כעת על העץ המתאים לקידוד. שם פונקציית המטרה היא $\sum_{\sigma \in \Sigma} f_\sigma \cdot h_\sigma$ כאשר h זה העומק בעץ. קל לראות שהפונקציות שקולות.

שלוש אבחנות

- **אבחנה ראשונה** – בעץ האופטימלי העלה הכי עמוק יהיה עם התדירות הכי נמוכה.

שלוש אבחנות

- **אבחנה ראשונה** – בעץ האופטימלי העלה הכי עמוק יהיה עם התדירות הכי נמוכה.
- נשכנע את עצמנו על ידי טיעון החלפה פשוט – אחרת, היה אפשר להחליף את העלה הכי עמוק ואת העלה עם התדירות הכי נמוכה, ורק להקטין את פונקציית המטרה. לכן בה"כ העלה הכי עמוק עם התדירות הכי נמוכה.

שלוש אבחנות

- **אבחנה ראשונה** – בעץ האופטימלי העלה הכי עמוק יהיה עם התדירות הכי נמוכה.
- נשכנע את עצמנו על ידי טיעון החלפה פשוט – אחרת, היה אפשר להחליף את העלה הכי עמוק ואת העלה עם התדירות הכי נמוכה, ורק להקטין את פונקציית המטרה. לכן בה"כ העלה הכי עמוק עם התדירות הכי נמוכה.
- **אבחנה שנייה** – בעץ האופטימלי לכל צומת פנימית יש שני בנים.

שלוש אבחנות

- **אבחנה ראשונה** – בעץ האופטימלי העלה הכי עמוק יהיה עם התדירות הכי נמוכה.
- נשכנע את עצמנו על ידי טיעון החלפה פשוט – אחרת, היה אפשר להחליף את העלה הכי עמוק ואת העלה עם התדירות הכי נמוכה, ורק להקטין את פונקציית המטרה. לכן בה"כ העלה הכי עמוק עם התדירות הכי נמוכה.
- **אבחנה שנייה** – בעץ האופטימלי לכל צומת פנימית יש שני בנים.
- אחרת, נאחד את האבא עם הבן הבודד שלו, לא הרסנו את תכונת חוסר הרישות, ויש לנו קוד יותר יעיל. כלומר, בה"כ לכל צומת פנימית יש שני בנים.

שלוש אבחנות

- **אבחנה ראשונה** – בעץ האופטימלי העלה הכי עמוק יהיה עם התדירות הכי נמוכה.
- נשכנע את עצמנו על ידי טיעון החלפה פשוט – אחרת, היה אפשר להחליף את העלה הכי עמוק ואת העלה עם התדירות הכי נמוכה, ורק להקטין את פונקציית המטרה. לכן בה"כ העלה הכי עמוק עם התדירות הכי נמוכה.
- **אבחנה שנייה** – בעץ האופטימלי לכל צומת פנימית יש שני בנים.
- אחרת, נאחד את האבא עם הבן הבודד שלו, לא הרסנו את תכונת חוסר הרישות, ויש לנו קוד יותר יעיל. כלומר, בה"כ לכל צומת פנימית יש שני בנים.
- **אבחנה שלישית** – שתי הצמתים עם תדירויות מינימליות הן אחיות בעץ. ההוכחה דומה להוכחה הראשונה.

האלגוריתם

1. נבנה עץ בינארי עם התווים בתור עלים.
2. כל אבא עם בן יחיד, נאחד עם הבן
3. נחליף את שני האחים עם הרמה הכי נמוכה בשני הצמתים עם התדירות הכי נמוכה.
4. נלך לשתי הצמתים עם התדירות הכי נמוכה, נאחד אותן (ואת האבא שלהן) לצומת אחת עם תדירות=סכום התדירויות.
5. נריץ שוב את שלב שלוש, עד שנגיע לצומת אחת.
6. נפצל כל צומת לשני הצמתים שמרכיבים אותה. אם אין כאלה נדלג עליה.

האלגוריתם

1. נבנה עץ בינארי עם התווים בתור עלים.
2. כל אבא עם בן יחיד, נאחד עם הבן
3. נחליף את שני האחים עם הרמה הכי נמוכה בשני הצמתים עם התדירות הכי נמוכה.
4. נלך לשתי הצמתים עם התדירות הכי נמוכה, נאחד אותן (ואת האבא שלהן) לצומת אחת עם תדירות=סכום התדירויות.
5. נריץ שוב את שלב שלוש, עד שנגיע לצומת אחת.
6. נפצל כל צומת לשני הצמתים שמרכיבים אותה. אם אין כאלה נדלג עליה.

• נציג על הלוח דוגמא לטקסט $T = aaeabbbccdd$.

מימוש נוסף

- מימוש נוסף לאלגוריתם הוא כדלקמן:
 1. בנה ערמת מינימום, בתוכה כל התווים עם התדירויות.
 2. הוצא את שני המינימומים, והכנס את הסכום שלהם.
 3. חזור עד שיש איבר אחד בערימה.
 4. בונים את העץ המתאים (תרגיל – כתבו פורמלי)
- מהמימוש הזה ניתן לראות שהבנייה של הקוד היא בזמן $O(n \log n)$.

אופטימליות הפתרון

- נוכיח שבכל צעד אנחנו ממשיכים למקסם את פונקציית המטרה.
- נניח שבשלב מסוים התדירויות המינימליות הן f_i, f_j . לאחר השלב הזה הצמתים יאוחדו. תהי F פונקציית המטרה לפני האיחוד, ו- F^* פונקציית המטרה לאחר האיחוד. קל לראות שמתקיים $F - F^* = f_i + f_j$.
- f_i, f_j קבועים, ולכן הפונקציות נבדלות בקבוע. כלומר, למקסם את F שקול ללמקסם את F^* .

אופטימליות הפתרון

- נוכיח שבכל צעד אנחנו ממשיכים למקסם את פונקציית המטרה.
- נניח שבשלב מסוים התדירויות המינימליות הן f_i, f_j . לאחר השלב הזה הצמתים יאוחדו. תהי F פונקציית המטרה לפני האיחוד, ו- F^* פונקציית המטרה לאחר האיחוד. קל לראות שמתקיים $F - F^* = f_i + f_j$.
- f_i, f_j קבועים, ולכן הפונקציות נבדלות בקבוע. כלומר, למקסם את F שקול ללמקסם את F^* .
- לכן, בכל צעד אנחנו ממשיכים למקסם את פונקציית המטרה המקורית.
- בכך הוכחנו שהקידוד שיוצא הוא אופטימלי.

תת מערך גדול ביותר

The Closest Pair Problem

בעיית תת מערך גדול ביותר

- קלט: מערך $A[1, \dots, n]$ מספרים שלמים שונים.
- פלט: תת מערך $A[i, \dots, j]$ כך שסכום איבריו הוא הגדול ביותר האפשרי.
- נקרא לתת מערך כזה האינטרוול הכבד ביותר המערך.

לדוגמה: עבור המערך הבא:

0	1	2	3	4	5	6	7	8	9
1	1	-2	3	5	-5	6	-10	-3	8

0	1	2	3	4	5	6	7	8	9
1	1	-2	3	5	-5	6	-10	-3	8

האינטרוול הכבד ביותר היינו $A[3, \dots, 6]$, אשר סכומו היינו 9.

פתרון ראשון:

- ספקו פיתרון בסיבוכיות $O(n \log n)$

פתרון ראשון:

- ספקו פיתרון בסיבוכיות $O(n \log n)$
- פיתרון:
- חלק את המערך ל-2 חלקים, $A[1, \dots, \frac{n}{2}]$, $A[\frac{n}{2} + 1, \dots, n]$.
- מצא תת מערך מקסימלי בכל אחד מהם (כמובן רקורסיה ממשיכה).
- מצא תת מערך בעל סכום גדול ביותר ב- $A[\frac{n}{2} + 1, \dots, n]$ שמתחיל ב- $\frac{n}{2} + 1$.
- מצא תת מערך בעל סכום גדול ביותר ב- $A[1, \dots, \frac{n}{2}]$ שמתחיל ב-1.
- חבר את סכום תת המערכים הללו.
- החזר את המקס' בין שלושת תת המערכים שקיבלת.

סיבוכיות+נכונות

- ברור כי עומק הרקורסיה הוא $O(\log n)$
- כל שלב ברקורסיה אנו עושים פעולות שלוקחות $O(n)$ זמן.
- השוואה בין סכומים של תתי מערכים לוקחת זמן קבוע, למצוא את התת מערך השלישי לוקחת זמן ליניארי (ניתן לעשות זאת עם מונה של סכום).
- לכן סה"כ סיבוכיות $O(n \log n)$
- נכונות: תנסו לעשות באינדוקציה (פשוטה יחסית ודיי ברורה).
- תר' בונוס: נסו לחשוב על פיתרון ב- $O(n)$

